Evaluating AI Agents

Observing Agents





DeepLearning.AI

Talking Head

In this lesson:

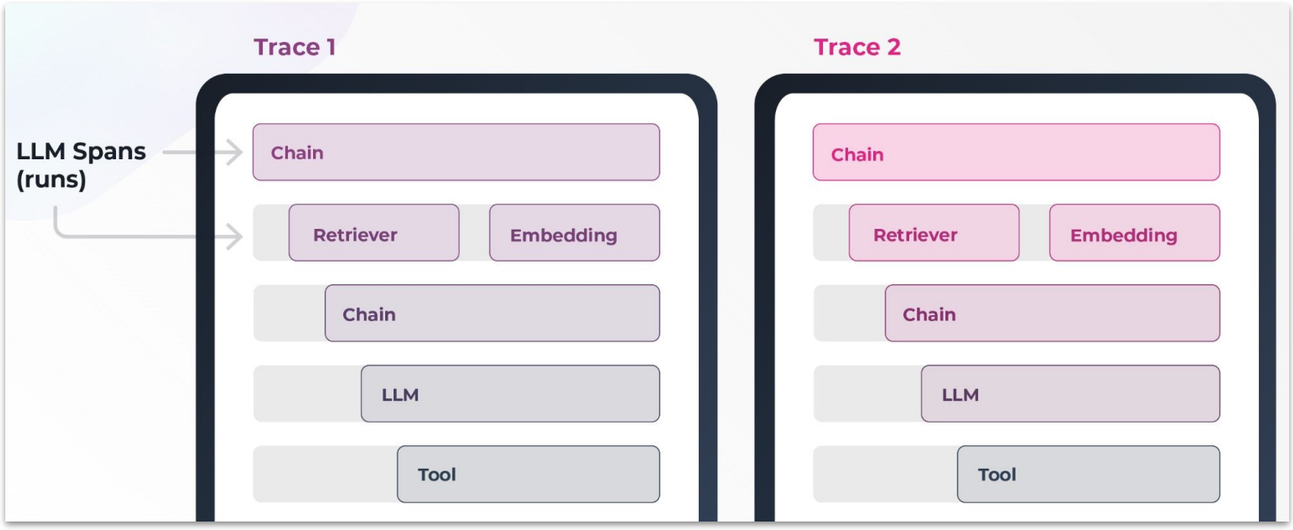
- Adding tracing and observability to an agent

Observing your Agent

Observability - complete visibility into every layer of an LLM-based software system: the application, the prompt, and the response.

Traces:

- A trace records the paths taken by requests (made by an application or end-user) as they propagate through multiple steps.

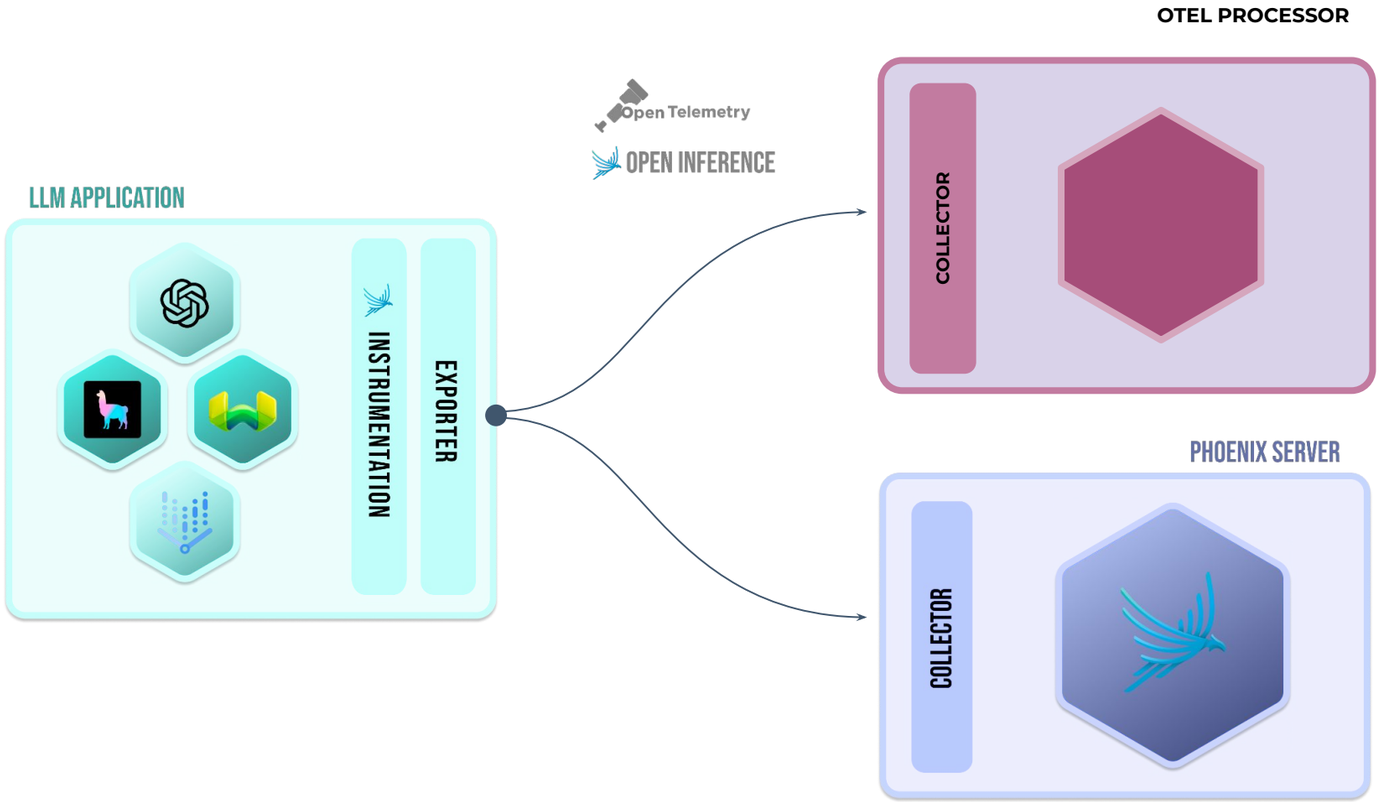


Spans:

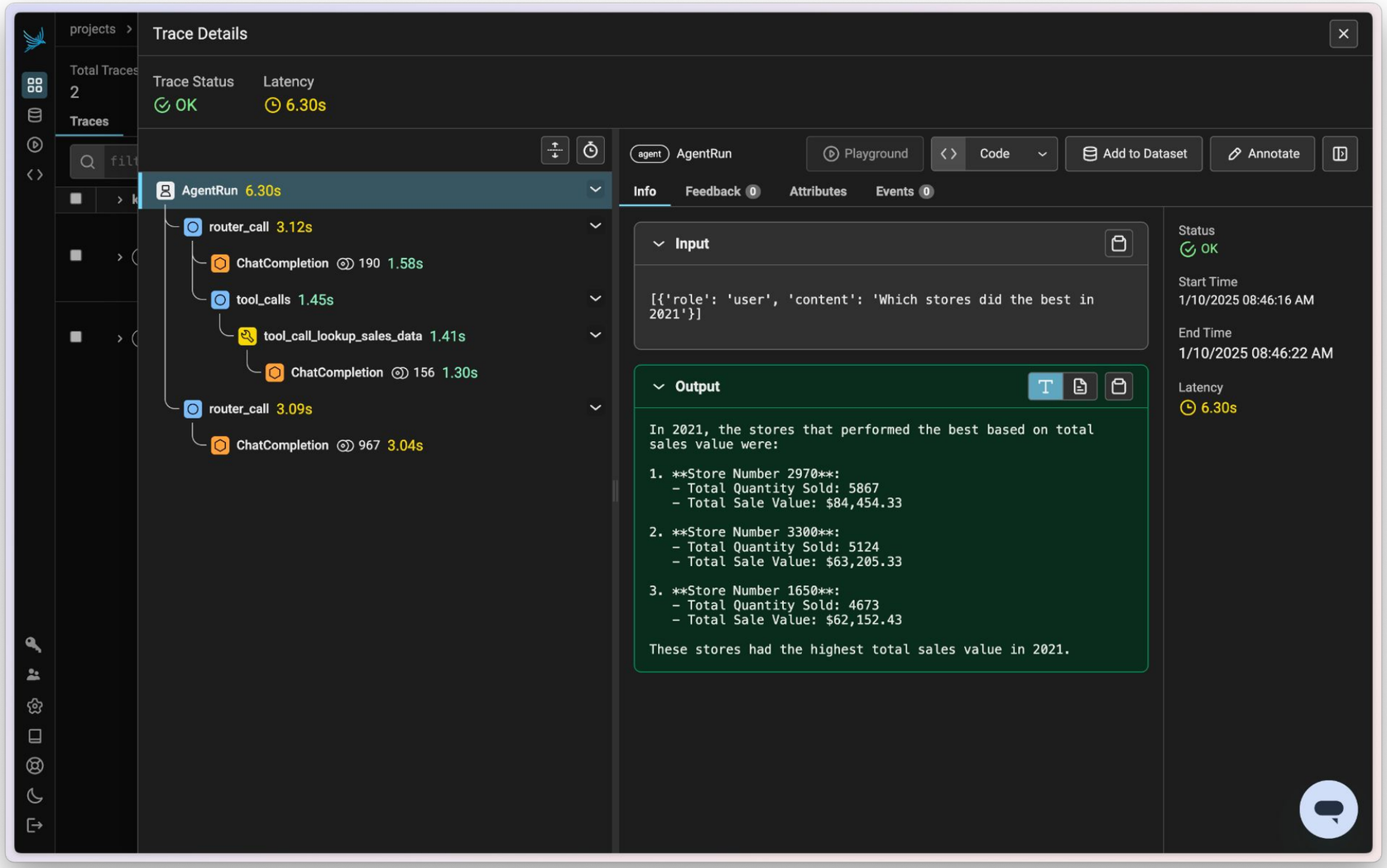
- Data captured on individual steps in an LLM app or pipeline.

OpenTelemetry (OTEL)

"OTEL" is one of the most widely-used standards for application observability



An Example Trace in Arize Phoenix



Instrumentation

Marking from which functions or code blocks spans should be collected, and with which attributes

```
def do_work():
    with tracer.start_as_current_span("span-name") as span:
        # do some work that 'span' will track
        print("doing some work...")
        # When the 'with' block goes out of scope, 'span' is closed for you
```

Arize Phoenix automates this process for you

Why Observability is Important

1. **Simplifies debugging** when building your application
2. **Provides a detailed log** of each call made in your app, which is necessary to run performance evaluations
3. Helps you understand, and soon control, the **unpredictable behavior of LLMs**

What you've learned:

1. What Observability is, how it works, and why it's important
2. The building blocks of Observability: Traces and Spans
3. How traces and spans are collected